

School of Engineering, Applied Sciences and Technology

Independent Set — NP-Completeness Analysis

Algorithmic Analysis and Reduction

Student: Leanne Jessica Rodrigo

Student ID: 20210001983

Course: BCS 309 — Algorithms I

Instructor: Dr. Arash Kermani

Submission Date: April 28, 2026

Contents

1	Introduction	1
2	Problem Statement	1
3	Asymptotic Analysis	2
3.1	Brute-force Algorithm	2
3.2	Polynomial-time Verifier	3
3.3	Non-trivial Algorithm	3
3.4	Comparison of Approaches	4
4	Alternate Polynomial-Time Reduction	5
4.1	Source Problem	5
4.2	Reduction	5
4.3	Correctness	6
4.4	Time Complexity	7
5	Conclusion	7

1. Introduction

The **Independent Set** problem asks whether a graph contains a sufficiently large collection of vertices with no edges between any chosen pair. This problem is central in introductory complexity analysis because it combines a simple graph-based definition with computational behavior that quickly becomes difficult as the graph grows. In the language of decision problems, the input is a graph together with a target size, and the goal is to answer only *YES* or *NO*. That viewpoint makes the problem suitable for brute-force analysis, polynomial-time verification, heuristic design, and reduction-based reasoning.

The explanations in this report are guided by the [*Independent Set Visualizer*](#), which illustrates how candidate independent sets behave, how a greedy selection strategy can proceed by favoring low-degree vertices, and how graph structure affects the final answer. Throughout this report, the visualizer is used only as an intuition-building aid for the algorithms discussed here.

This report covers four main tasks. First, it defines the Independent Set decision problem and gives simple YES and NO examples. Next, it analyzes three algorithmic viewpoints: brute force, a polynomial-time verifier, and a greedy heuristic consistent with the visualizer. Finally, it presents a polynomial-time reduction from **Clique** to **Independent Set** using the complement graph, including a complete worked example and a proof of correctness.

2. Problem Statement

The problem **Independent Set** is defined as follows. The input is an undirected graph $G = (V, E)$ and an integer k . The question is: *does there exist a subset $S \subseteq V$ such that $|S| \geq k$ and no two distinct vertices in S are joined by an edge of G ?* If such a set exists, the answer is **YES**; otherwise, the answer is **NO**.

A YES example is the path graph with vertices $V = \{v_1, v_2, v_3, v_4\}$ and edges $E = \{\{v_1, v_2\}, \{v_2, v_3\}, \{v_3, v_4\}\}$, with $k = 2$. The set $\{v_1, v_3\}$ is independent because v_1 and v_3 are not adjacent, so the instance is a YES-instance. A NO example is the triangle graph with vertices $V = \{a, b, c\}$ and edges $E = \{\{a, b\}, \{b, c\}, \{a, c\}\}$, with $k = 2$. Every pair of vertices is adjacent, so there is no independent set of size 2, and the answer is NO.

3. Asymptotic Analysis

3.1 Brute-force Algorithm

A brute-force solution checks every possible subset of vertices and asks whether any such subset is an independent set of size at least k .

Pseudocode

```
1 BruteForceIS (G=(V,E) , k)
2 for each subset S of V do
3     if |S| >= k then
4         if S contains no adjacent pair then
5             return YES
6 return NO
```

To derive the running time, let $n = |V|$. Each vertex has two possibilities in a candidate subset: it is either included or excluded. Therefore the total number of subsets of V is

$$2 \cdot 2 \cdots 2 = 2^n,$$

with one factor of 2 for each of the n vertices. Hence the outer enumeration alone already requires 2^n iterations.

For each subset S , the algorithm must test whether two chosen vertices are adjacent. In the worst case, the subset can contain all n vertices, so checking all unordered pairs takes at most

$$\binom{n}{2} = \frac{n(n-1)}{2} = O(n^2).$$

Multiplying the number of subsets by the cost per subset gives the total running time:

$$T(n) = 2^n \cdot O(n^2) = O(2^n n^2).$$

This derivation shows explicitly why the algorithm is exponential: the term 2^n comes from counting all subsets.

The space complexity is polynomial. The graph itself requires either $O(n^2)$ space with an adjacency matrix or $O(n + m)$ with adjacency lists, where $m = |E|$. The current subset can be stored using an n -bit indicator vector or a list of selected vertices, which adds at most $O(n)$ extra space. Thus the total space usage is polynomial in the input size.

3.2 Polynomial-time Verifier

A polynomial-time verifier receives a certificate claiming that the instance is a YES-instance and checks that claim efficiently.

Certificate. A certificate is a subset $C \subseteq V$ presented as the claimed independent set. The verifier must confirm two facts: first, that $|C| \geq k$; second, that no two distinct vertices in C are adjacent in G .

Verifier pseudocode

```
1 VerifyIS (G=(V,E), k, C)
2 if |C| < k then return REJECT
3 for each distinct pair u,v in C do
4     if {u,v} is an edge of G then
5         return REJECT
6 return ACCEPT
```

Now derive the running time in terms of $n = |V|$. The size test $|C| \geq k$ takes constant time once the certificate is represented. The pairwise check examines every unordered pair in C . If $|C| = r$, then the number of such pairs is

$$\binom{r}{2} = \frac{r(r-1)}{2}.$$

Since $r \leq n$, this quantity is at most

$$\binom{n}{2} = \frac{n(n-1)}{2} = \Theta(n^2).$$

With constant-time edge lookup using an adjacency matrix, the verifier runs in

$$\Theta(n^2)$$

in the worst case. This bound is tight because when the certificate contains nearly all vertices, the verifier really does need to inspect on the order of n^2 pairs.

The space complexity is again polynomial. Besides storing the input graph, the verifier stores the certificate and a small number of loop variables, so the additional working space is $O(n)$.

3.3 Non-trivial Algorithm

A more informative algorithm is a greedy heuristic consistent with the behavior suggested by the visualizer. The visualizer naturally motivates choosing a vertex of small degree,

because such a vertex conflicts with relatively few others. After selecting that vertex, the algorithm deletes it and all of its neighbors, since none of those neighbors can coexist with the chosen vertex in an independent set.

Pseudocode

```

1 GreedyMinDegreeIS (G=(V,E) , k)
2 S ← empty set
3 while V is not empty do
4     choose a vertex v of minimum degree
5     add v to S
6     delete v from G
7     delete every neighbor of v from G
8 if |S| ≥ k then
9     return YES
10 return NO

```

To derive the running time, let the current graph have at most n vertices. In a straightforward implementation, each iteration may scan the current graph to compute or compare degrees, which costs at most $O(n^2)$ in a dense graph. At least one vertex is removed in every iteration, so the loop executes at most n times. Therefore,

$$T(n) \leq n \cdot O(n^2) = O(n^3).$$

This bound is polynomial and follows directly from the fact that there are at most n rounds, each with at most quadratic work.

The space complexity is polynomial for the same reason as before: the graph requires $O(n^2)$ space with an adjacency matrix or $O(n + m)$ with adjacency lists, and the selected set S uses at most $O(n)$ extra space.

From an approximation perspective, the greedy method is useful because it always produces a *valid* independent set and often builds a reasonably large one quickly. The intuition seen in the visualizer is that choosing a low-degree vertex tends to eliminate fewer future options than choosing a high-degree vertex. However, this is only a heuristic. A locally attractive choice can still prevent a larger independent set from being formed later, so the algorithm does not guarantee an optimal answer on every graph. For that reason, it should be viewed as a fast constructive method rather than an exact optimization algorithm.

3.4 Comparison of Approaches

The following table summarizes the differences between the three approaches in terms of complexity, correctness, and practical applicability.

Aspect	Brute Force	Verifier	Greedy
Time Complexity	$O(2^n n^2)$	$\Theta(n^2)$	$O(n^3)$
Correctness	Always optimal	Always correct	Not guaranteed optimal
Purpose	Exact solution	Verification	Fast heuristic
Practical Use	Only feasible for small n	Confirms NP-membership	Efficient heuristic for large graphs

4. Alternate Polynomial-Time Reduction

4.1 Source Problem

The source problem is **Clique**. Given an undirected graph $G = (V, E)$ and an integer k , the Clique decision problem asks whether there exists a subset $C \subseteq V$ such that $|C| \geq k$ and every two distinct vertices in C are adjacent. **This differs from the source problem used in the final project, which was 3-SAT.**

4.2 Reduction

Define the reduction function by

$$f(G, k) = (\overline{G}, k),$$

where $\overline{G}$ is the complement of G . The complement graph keeps the same vertex set and reverses adjacency between every pair of distinct vertices. Formally, if $G = (V, E)$, then

$$\overline{G} = (V, \overline{E}), \quad \{u, v\} \in \overline{E} \text{ if and only if } \{u, v\} \notin E$$

for all distinct vertices $u, v \in V$.

Worked example. Consider the Clique instance $(G, 3)$ where

$$V = \{1, 2, 3, 4\}$$

and

$$E = \{\{1, 2\}, \{1, 3\}, \{2, 3\}, \{2, 4\}\}.$$

In the original graph, the vertices $\{1, 2, 3\}$ form a clique of size 3 because all three edges $\{1, 2\}$, $\{1, 3\}$, and $\{2, 3\}$ are present.

Now construct the complement graph step by step by examining every unordered pair of

distinct vertices:

$$\begin{array}{ll}
\{1, 2\} \in E, & \text{so } \{1, 2\} \notin \overline{E}, \\
\{1, 3\} \in E, & \text{so } \{1, 3\} \notin \overline{E}, \\
\{1, 4\} \notin E, & \text{so } \{1, 4\} \in \overline{E}, \\
\{2, 3\} \in E, & \text{so } \{2, 3\} \notin \overline{E}, \\
\{2, 4\} \in E, & \text{so } \{2, 4\} \notin \overline{E}, \\
\{3, 4\} \notin E, & \text{so } \{3, 4\} \in \overline{E}.
\end{array}$$

Therefore the complement graph is

$$\overline{G} = (V, \overline{E}) \quad \text{with} \quad \overline{E} = \{\{1, 4\}, \{3, 4\}\}.$$

Hence the reduction outputs the Independent Set instance

$$(\overline{G}, 3).$$

To see the result clearly, observe that in $\overline{G}$ the vertices 1, 2, and 3 have no edges among them. That means $\{1, 2, 3\}$ is an independent set of size 3 in the complement graph. The worked example therefore demonstrates exactly how a clique in G becomes an independent set in $\overline{G}$.

4.3 Correctness

The reduction is correct if and only if (G, k) is a YES-instance of Clique exactly when $(\overline{G}, k)$ is a YES-instance of Independent Set.

Direction 1: Clique $\Rightarrow$ Independent Set. Assume (G, k) is a YES-instance of Clique. Then there exists a set $C \subseteq V$ with $|C| \geq k$ such that every two distinct vertices in C are adjacent in G . Take any distinct vertices $u, v \in C$. Because C is a clique, $\{u, v\} \in E$. By the definition of the complement graph, this implies $\{u, v\} \notin \overline{E}$. Since this is true for every distinct pair in C , no two vertices of C are adjacent in $\overline{G}$. Therefore C is an independent set in $\overline{G}$ of size at least k , so $(\overline{G}, k)$ is a YES-instance of Independent Set.

Direction 2: Independent Set $\Rightarrow$ Clique. Assume $(\overline{G}, k)$ is a YES-instance of Independent Set. Then there exists a set $S \subseteq V$ with $|S| \geq k$ such that no two distinct vertices in S are adjacent in $\overline{G}$. Take any distinct vertices $u, v \in S$. Because S is independent in $\overline{G}$, we have $\{u, v\} \notin \overline{E}$. By the definition of the complement graph, this means $\{u, v\} \in E$. Since every distinct pair in S is adjacent in G , the set S is a clique in G of size at least k . Therefore (G, k) is a YES-instance of Clique.

Because both directions hold, the reduction is correct.

4.4 Time Complexity

Now derive the complexity of constructing $\overline{G}$. Let $n = |V|$. The complement graph is determined by checking every unordered pair of distinct vertices. The number of such pairs is

$$\binom{n}{2} = \frac{n(n-1)}{2}.$$

For each pair $\{u, v\}$, the construction performs a constant-time decision: if the edge is present in G , it is absent in $\overline{G}$; if it is absent in G , it is present in $\overline{G}$. Therefore the total running time is proportional to the number of pairs:

$$T(n) = O\left(\binom{n}{2}\right) = O(n^2).$$

Thus the reduction function f runs in polynomial time.

5. Conclusion

This report has analyzed Independent Set from both algorithmic and complexity-theoretic perspectives. The brute-force method demonstrates why the problem becomes difficult quickly, since the number of vertex subsets grows as 2^n . The verifier shows that once a candidate set is provided, correctness can be checked in polynomial time by testing pairwise non-adjacency. The greedy min-degree heuristic, motivated by the behavior visible in the visualizer, provides a practical way to construct independent sets efficiently even though it does not always produce an optimal one.

The reduction from Clique to Independent Set reveals a precise structural relationship between the two problems. A clique in a graph becomes an independent set in the complement graph, and an independent set in the complement becomes a clique in the original graph. This equivalence explains why the two problems mirror each other so naturally. More broadly, the analysis illustrates a core theme of computational complexity: some problems are easy to verify and easy to transform into one another, yet difficult to solve exactly by exhaustive search as input size increases.